Network Intrusion Detection System with Explainable AI

Kotagiri Charan Kumar
Roll No. 160923750301

Mohammed Abdul Kareem
Roll No. 160923750303

Mohammed Mujtaba Quadri
Roll No. 160923750304

Department of Computer Science and Engineering (Data Science)
Lords Institute of Engineering and Technology, Hyderabad, India
Guide: Ms. Zoya Tabassum

***Abstract*—Network intrusion detection systems (NIDS) based on machine learning reach high detection rates, but most of them report an alert without explaining it. This limits their use in small organisations that have no full-time security analyst. This paper presents a Network Intrusion Detection System with Explainable AI that protects a web server or a small-business network against four attack types: port scan, brute force, denial of service (DoS) and distributed denial of service (DDoS). The system is trained on the CICIDS2017 dataset with Random Forest and XGBoost, and the model with the higher macro F1-score is kept; XGBoost reached a macro F1-score of 0.9998. For every detected attack, the system reports the attack type, a confidence value, the time of detection, a plain-English explanation built from SHAP values, and a recommended action. A Live Mode connects an attacker laptop and a detector laptop over a network, so that attacks sent from one computer are detected and explained on the other in real time on a web dashboard.**

***Index Terms*—network intrusion detection, explainable AI, SHAP, XGBoost, Random Forest, CICIDS2017, cybersecurity.**

I. Introduction

Small businesses depend more and more on their own websites and servers, and these are scanned, probed and attacked from the internet all the time. A network intrusion detection system (NIDS) watches network traffic and raises an alert when it finds bad behaviour such as port scanning, password guessing or flooding. Machine learning models can find such behaviour with high accuracy, and deep and ensemble models often give the best results on benchmark data [1], [4].

The difficulty is trust. A model that only prints “attack” gives the owner of a small business nothing to act on, and this lack of transparency is one reason why complex models are rarely used in practice [4]. Explainable AI (XAI) methods help by showing which input features pushed a prediction towards a class. Among them, SHapley Additive exPlanations (SHAP) [2] is widely used because it has a strong theoretical base. However, raw feature-importance values still need machine-learning knowledge to read [5].

This paper presents a Network Intrusion Detection System with Explainable AI that turns each detection into an alert that a non-expert can read and act on. The main contributions are: (i) a classifier for normal traffic and four attack classes trained on CICIDS2017, comparing Random Forest and XGBoost; (ii) an alert format that combines attack type, confidence, time of detection, a SHAP-based plain-English explanation and a recommended action; (iii) a Live Mode in which one computer sends attacks and a second computer detects and explains them in real time; and (iv) a web dashboard that shows the alerts. The rest of the paper is organised as follows. Section II states the problem, Section III reviews related work, Sections IV and V describe the data and the proposed system, Sections VI and VII give the evaluation and results, and Sections VIII and IX discuss limits and conclude.

II. Problem Statement and Objectives

Many micro, small and medium enterprises (MSMEs) run their own website or server but cannot pay for a security operations team. The person who receives an alert is often the owner or a general IT person. This reader needs to know what happened, how sure the system is, why the system thinks so, and what to do next. Existing work either gives only a class label or gives explanations that need expert knowledge (Section III).

The objectives of this work are: (i) to detect normal traffic and four attack classes, using the macro F1-score as the main measure; (ii) to compare Random Forest and XGBoost fairly and keep the better model; (iii) to explain every alert in plain English using SHAP; (iv) to suggest a next step for each attack type; and (v) to show the full pipeline working across two computers in Live Mode.

III. Related Work

Sharafaldin et al. created the CICIDS2017 dataset in a testbed with separate attacker and victim networks, and extracted more than 80 flow features with CICFlowMeter. It contains normal traffic and attacks such as brute force, DoS, DDoS, port scan, web attacks, infiltration, botnet and Heartbleed. In their tests of seven machine-learning algorithms, Random Forest trained the fastest while keeping the highest accuracy [1].

Lundberg and Lee introduced SHAP, which gives each feature an importance value for one prediction. They showed that, among additive feature attribution methods, one unique solution satisfies local accuracy, missingness and consistency [2]. Lundberg et al. later developed a fast and exact algorithm for tree ensembles [3], which is the form used in this work.

Mane and Rao built an explainable-AI framework for NIDS on a deep neural network trained on the NSL-KDD dataset. They used SHAP, LIME, contrastive explanations, ProtoDash and Boolean rules, and argued that data scientists, analysts and end users each need a different kind of explanation [4]. Houssel et al. proposed eX-NIDS, which gives the flows flagged by a NIDS, together with context such as IP history and protocol names, to a large language model that writes the explanation. Adding this context raised the correctness of GPT-4 explanations from 40% to 80%,

but a locally hosted Llama 3 model needed about four to six seconds per explanation [5]. Table I summarises these studies.

TABLE I
SUMMARY OF RELATED WORK

Study	Year	Approach	Limitation
Sharafaldin et al. [1]	2018	CICIDS2017 dataset; seven ML algorithms compared	Dataset paper; alerts are not explained
Lundberg et al. [2], [3]	2017, 2020	SHAP and TreeSHAP for model explanation	Numeric values need ML knowledge to read [5]
Mane and Rao [4]	2021	Deep neural network on NSL-KDD with five XAI methods	Older dataset; accuracy of 82.4%
Houssel et al. [5]	2025	eX-NIDS: a language model writes the explanation	Heavy model; 4–6 s per explanation locally; errors if context is missing

In contrast to these works, our system uses fast tree-based models on CICIDS2017, turns SHAP values into short plain-English reasons, adds a recommended action, and shows the full pipeline across two machines.

IV. DATASET AND PREPROCESSING

A. CICIDS2017

CICIDS2017 was captured over five days in July 2017. Monday has only normal traffic, and the attacks were run on the other days with publicly available tools [1]. The traffic is stored as labelled flows described by statistical features such as flow duration, packet counts, packet lengths, inter-arrival times and TCP flag counts. Table II shows the five classes used in this work and the tools that made them [1]. The other attack types in the dataset (web attacks, infiltration, botnet and Heartbleed) are outside the scope of this work.

TABLE II
CLASSES USED AND ATTACK TOOLS IN CICIDS2017

Class	Tools used to generate the traffic
Normal	Normal user profiles (HTTP, HTTPS, FTP, SSH, e-mail)
PortScan	Nmap with several scan types
BruteForce	FTP-Patator and SSH-Patator
DoS	Hulk, GoldenEye, Slowloris, Slowhttptest
DDoS	LOIC

B. Preprocessing

The eight daily CICIDS2017 files were combined and the attack labels were mapped to the five classes above. Identifier columns such as flow ID, IP addresses and timestamps were removed, so that the model learns the behaviour of the traffic and not the identity of particular hosts. Infinite and missing values were replaced with zero. Because the classes are strongly imbalanced, a stratified sample of at most 500,000 flows was drawn and divided into 80% training data and 20% test data, keeping the class proportions.

V. PROPOSED SYSTEM

A. System Architecture

The system has two parts (Fig. 1). In the offline part, the data are prepared, two models are trained, and the one with the higher macro F1-score is saved. In the online part, every traffic record goes through the same ordered steps: the classifier gives the attack type and a confidence value, the time of detection is recorded, a SHAP explanation is made, and a recommended action is looked up last. The result is shown as an alert card on the dashboard.

Three simple rules guided the design. First, every alert must answer the same five questions in the same order, so that the reader learns the card once. Second, the explanation must be short and in plain words, never a list of numbers. Third, the detection step and the advice step are kept apart: the model decides what the traffic is, and a fixed rule table decides what to suggest.

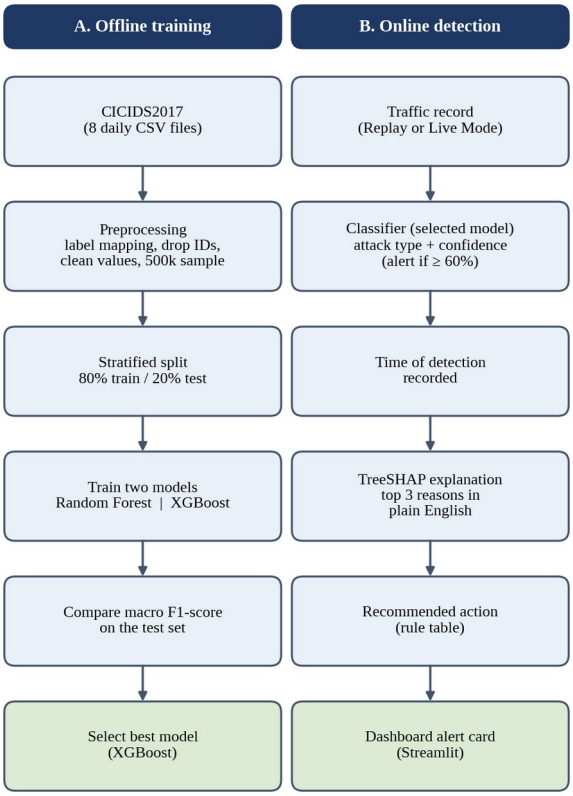


Fig. 1. System architecture: offline training (A) and online detection (B).

B. Classification Models

Two tree-based models were trained. Random Forest [6] is an ensemble of decision trees, each trained on a random part of the data and features, and it gives its answer by voting. XGBoost [7] is a gradient-boosted tree ensemble in which each new tree corrects the errors of the earlier trees. Both are fast on tabular data and work directly with TreeSHAP. Because normal traffic is the largest class, accuracy alone would be misleading, so the model with the higher macro-averaged F1-score is selected. The models were built with scikit-learn [8] and the xgboost library.

TABLE III
RANDOM FOREST AND XGBOOST COMPARED

Point	Random Forest	XGBoost
Tree building	Many trees built independently on random parts of the data	Trees built one after another; each corrects earlier errors
Final answer	Vote of all trees	Sum of the scores of all trees
Works with TreeSHAP	Yes	Yes
Role in this work	Candidate model	Candidate model (selected)

C. Confidence Value and Alert Rule

The confidence of an alert is the probability that the model gives to the predicted class, shown as a percentage. It is a raw prediction probability and not a calibrated statistical confidence. A flow is reported as an attack only when the predicted class is not Normal and the confidence is at least 60%.

D. SHAP-Based Explanation

For every flow reported as an attack, TreeSHAP [2], [3] computes how much each feature contributed to the predicted class. For a feature i in the feature set F, the SHAP value is the weighted average of its effect over all feature subsets S:

$$\varphi_i = \Sigma_{S \subseteq F \setminus \{i\}} \, w(S) \, [\, f(S \cup \{i\}) - f(S) \,] \qquad (1)$$

where $w(S) = |S|! \, (|F| - |S| - 1)! \, / \, |F|!$ and $f(S)$ is the model output when only the features in S are known. A positive value means that the feature pushed the prediction towards the attack class.

The three features with the largest positive contribution are taken and translated into short plain-English reasons through a fixed table of feature descriptions, for example a very high packet rate, many connections within a short time, or many small packets. These reasons are placed in the alert, so the reader sees the evidence behind the decision without reading numerical SHAP values.

E. Recommended Action

After the explanation, a rule table maps the attack type to a recommended response (Table IV). This table only suggests an action; it takes no part in detecting attacks.

TABLE IV
RECOMMENDED ACTIONS BY ATTACK TYPE

Attack type	Recommended action
PortScan	Block or rate-limit the source IP; close unused ports
BruteForce	Lock the targeted account; enable login-attempt limits
DoS	Rate-limit the source; restrict connections per client
DDoS	Enable upstream filtering; contact the hosting provider

F. Replay Mode and Live Mode

The same pipeline runs in two modes. In Replay Mode, held-out test records are passed through the pipeline one after another. Live Mode shows the system across two computers on the same network (Fig. 2). On the attacker laptop, a Python script sends attack traffic of a chosen type (port scan, brute force, DoS or DDoS) to the detector laptop over a TCP connection. The records are taken from CICIDS2017 test data that the model never saw during training, and their labels are removed before sending, so the detector must predict them. On the detector laptop, a receiver accepts the records and passes them to the model, the SHAP explainer and the rule table as they arrive. The source IP address in each alert is the address of the attacker laptop, taken from the network connection.

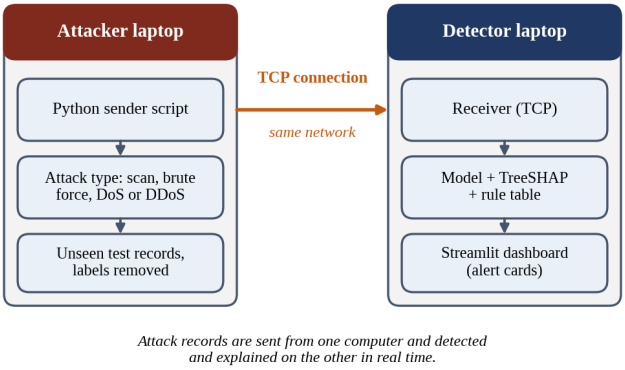


Fig. 2. Live Mode: attacks sent from one laptop are detected and explained on another.

To avoid flooding the reader, attacks of the same type from the same source IP that occur within a ten-second window are merged into one alert card with a flow count.

G. Dashboard

The detector laptop runs a web dashboard built with Streamlit. It shows counters for normal traffic and attack alerts, and one card per alert (Fig. 3). Each card shows the source and destination, the attack type and confidence, the time of detection, the SHAP-based reasons and the recommended action, in that order.

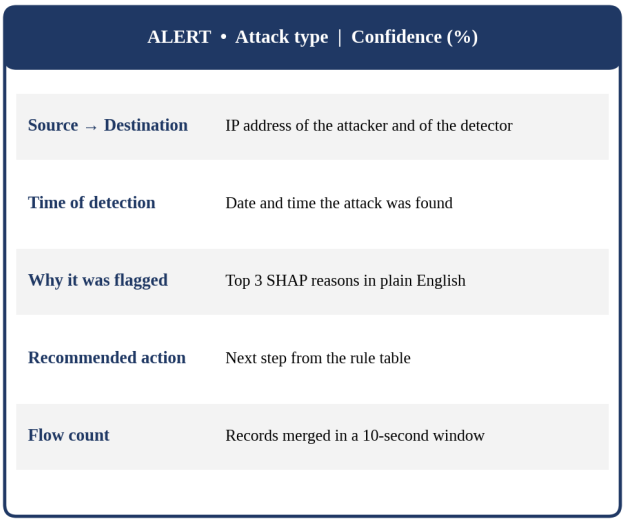


Fig. 3. Layout of one alert card on the dashboard.

H. Reading an Alert

An alert is meant to be read from top to bottom in a few seconds. For example, when a DDoS flood reaches the detector, the card names the attack type and the confidence, gives the time and the source IP address, lists three short reasons such as a very high packet rate, many connections in a short time and many small packets, and ends with the suggested step: enable upstream filtering and contact the hosting provider. The owner does not need to know what a SHAP value is, and still sees the evidence for the decision. SHAP is computed only for flows that pass the alert rule, which keeps the cost of explanation low on ordinary laptops.

VI. IMPLEMENTATION AND EVALUATION METHOD

The system was written in Python and run on ordinary laptops without a GPU. Table V lists the main components and their roles.

TABLE V
SOFTWARE COMPONENTS AND THEIR ROLES

Component	Role in the system
scikit-learn [8]	Random Forest, data split, evaluation
xgboost [7]	XGBoost classifier
SHAP [2], [3]	TreeSHAP feature contributions
Streamlit	Web dashboard and alert cards
TCP sockets	Sending and receiving records in Live Mode

Both models were trained on the same 80% training split and tested on the same 20% held-out split, so the comparison is fair. The test data were not used for training or for any choice of settings other than the final model selection by macro F1-score. In Live Mode, the records sent from the attacker laptop were taken from test data that the model never saw during training.

The measures used are precision, recall and F1-score for each class, and the macro-averaged F1-score, which gives every class the same weight and is the main figure reported in this paper. With *TP*, *FP* and *FN* as the true positives, false positives and false negatives of a class, and *C* as the number of classes (here five):

$$Precision = TP / (TP + FP) \quad (2)$$

$$Recall = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \quad (4)$$

$$Macro\ F1 = (1 / C)\ \Sigma\ F1_c,\ \ c = 1, \ldots, C \quad (5)$$

Table VI collects the main settings used in the experiments.

TABLE VI
SUMMARY OF EXPERIMENTAL SETTINGS

Setting	Value
Dataset	CICIDS2017 (eight daily files)
Classes	Normal, PortScan, BruteForce, DoS, DDoS
Sample size	At most 500,000 flows, stratified
Train / test split	80% / 20%, class proportions kept
Models compared	Random Forest and XGBoost
Selection measure	Macro F1-score on the test set
Alert rule	Not Normal and confidence ≥ 60%
Reasons per alert	Top 3 SHAP features
Merge window	10 s, same source IP and attack type
Hardware	Ordinary laptops, no GPU

VII. RESULTS

A. Model Selection

Random Forest and XGBoost were trained on the same training split and compared on the same held-out test data using the macro F1-score. XGBoost achieved a macro F1-score of 0.9998 and was selected as the final model. Because the macro F1-score gives the same weight to Normal traffic and to each of the four attack classes, this value shows that the selected model handles all five classes well, and not only the largest one.

B. Live Mode Evaluation

The Live Mode test follows five steps: (1) the dashboard and the receiver are started on the detector laptop; (2) the sender is started on the attacker laptop and an attack type is chosen; (3) the records are sent over the TCP connection with their labels removed; (4) the detector classifies each record and explains the attacks; and (5) the alert cards are checked against the attack type that was sent.

Each attack type (port scan, brute force, DoS and DDoS) was sent from the attacker laptop to the detector laptop together with a stream of normal traffic. The detector received the records, predicted their class, and showed an alert card with the attack type, confidence, time of detection, SHAP-based reasons and recommended action for each attack, with repeated alerts from the same source merged into one card. This shows that the full pipeline works across two computers.

TABLE VII
SUMMARY OF RESULTS

Item	Outcome
Selected model	XGBoost
Macro F1-score (test set)	0.9998
Classes handled	Normal, PortScan, BruteForce, DoS, DDoS
Live Mode	Full pipeline shown across two laptops
Alert content	Type, confidence, time, reasons, action

VIII. DISCUSSION AND LIMITATIONS

The alert format gives the reader what is needed to decide what to do: what happened, how sure the model is, when it happened, why the model thinks so, and what to do next. Tree-based models with TreeSHAP keep the explanation fast and repeatable, whereas explanations written by large language models depend on a heavy model and can contain errors if context is missing [5].

Several limits should be noted. First, all models are trained and tested on one dataset made in 2017 in a laboratory testbed, so high scores on this benchmark do not promise the same results on other networks or against newer attack tools. Second, a macro F1-score close to 1 on a random split of a benchmark should be read with care, because flows from the same attack run look alike and can fall in both the training and the test data. Third, Live Mode currently sends flow-feature records of the dataset to the detector; connecting it to real packet capture and flow extraction on the protected server is future work. Fourth, the confidence value is a raw model probability, and SHAP explains the behaviour of the model, which is not always the true cause of an attack. Fifth, slow or low-volume attacks may look like normal traffic. Sixth, per-class scores and Live Mode detection counts are not reported in this version and are left for the full test runs. Finally, the scope of this phase is limited to four attack types; alert correlation and mapping of alerts to attack frameworks such as MITRE ATT&CK were left out.

Live Mode is meant for a controlled demonstration. The two laptops belong to the project team and are on the same network, and what travels between them are dataset records, not attack packets aimed at any outside system. When the system is later connected to a real server, it should be used only on networks that the owner is allowed to monitor.

IX. CONCLUSION AND FUTURE WORK

This paper presented a Network Intrusion Detection System with Explainable AI that detects normal traffic, port

scan, brute force, DoS and DDoS attacks, and reports each detection with its type, confidence, time, a SHAP-based explanation in plain English and a recommended action. XGBoost reached a macro F1-score of 0.9998 on CICIDS2017, and Live Mode showed the complete pipeline working across an attacker laptop and a detector laptop. Future work includes real-time packet capture and flow extraction on the protected server, tests on more datasets such as CSE-CIC-IDS2018, a time-based split of the data, calibrated confidence values, and automatic response actions.

ACKNOWLEDGMENT

The authors thank their guide, Ms. Zoya Tabassum, and the Department of Computer Science and Engineering (Data Science), Lords Institute of Engineering and Technology, Hyderabad, for their guidance and support during this work.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[3] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[4] S. Mane and D. Rao, "Explaining network intrusion detection system using explainable AI framework," arXiv:2103.07110, 2021.

[5] P. R. B. Houssel, S. Layeghy, P. Singh, and M. Portmann, "eX-NIDS: A framework for explainable network intrusion detection leveraging large language models," arXiv:2507.16241, 2025.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.